Problem Set 7: Random Processes and Monte Carlo Methods

Readings: Newman Ch. 10

Due: Wednesday, December 2nd 11:59 PM

Instructions: Work through each problem, following the instructions for the requirements of the code and their outputs. Some starting code is provided for each problem on the course website. Problem sets should be submitted in one batch, with each problem uploaded as it's own python script (`.py` file) on the HuskyCT assignment page with the filename: `ps[PROBLEM SET NUMBER]_[QUESTION NUMBER]_[LASTNAME].py`.

Parameters:

(1) Submitted solutions are restricted to imports of external functions from standard built-in Python libraries (i.e. `math, time, os, sys`, etc.). In addition, imports from `matplotlib.pyplot` and `numpy` are permitted for basic array operations, unless otherwise stated.

(2) All plots should have both axes clearly labeled and include a title. Plots should include either legends or annotations for differentiating plot elements, when applicable.

---

1. RADIOACTIVE DECAY CHAIN

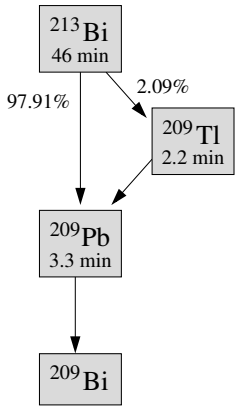


Figure 1: The isotope ^{213}Bi decays to stable ^{209}Bi via one of two different routes, with probabilities and half-lives given in this diagram

Starting with a sample initially consisting of $10\,000$ atoms of ^{213}Bi, write a program to simulate the decay chain of ^{213}Bi atoms. Adopt a timestep of $\Delta t = 1$ s. Your program should:

a. Calculate the probability for each decay step. Note, when a ^{213}Bi atom decays your program will have to also decide at random with the appropriate probability the route by which it decays.

b. Include a function that simulates the decay chain and keeps track of the number of atoms of each of the four isotopes at all times for $20\,000$ seconds.

c. Have your function print the total number of each isotope after 46 minutes using a random seed of 23434.

d. Plot the time evolution of all four isotopes on the same plot. Save this plot as `decay_[LASTNAME].png`

Hint: it is best to work up the chain from the bottom, not down from the top, to avoid inadvertently making the same atom decay twice on a single step.

2. VOLUME OF A HYPERSPHERE

Consider, the area of a circle of unit radius, given by the integral:

$$I = \iint_{-1}^{+1} f(x,y)dxdy$$

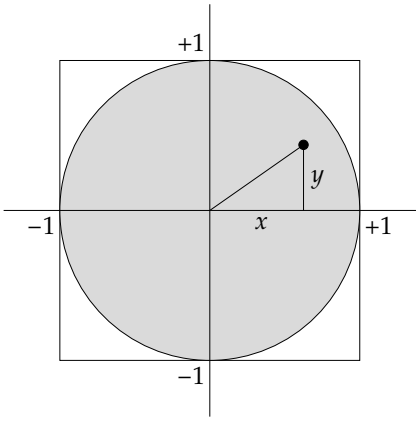


Figure 2: $f(x,y) = 1$ everywhere inside the circle and zero everywhere outside. In other words,

$$f(x,y) = \begin{cases} 1 & \text{if } x^2 + y^2 \leq 1, \\ 0 & \text{otherwise.} \end{cases}$$

If we were to calculate the area by Monte Carlo integration, we would generate a set of N random points (x,y), where both x and y are in the range from -1 to 1. Then the Monte Carlo integral for this calculation would be

$$I \simeq \frac{4}{N} \sum_{i=1}^{N} f(x_i, y_i).$$

Generalize this two-dimensional method to the ten-dimensional case and write a program to perform a Monte Carlo calculation of the volume of a sphere of unit radius in ten dimensions. Your program should:

a. Include a function to calculate the Monte-Carlo integral for an arbitrary number of dimensions d and points N.

b. Validate your generalization for the volume of a sphere, print your calculation and calculate it's error compared to the exact solution. Define an N that gets you $< 1\%$ error on this calculation.

c. Calculate and print the volume of a 10-D hypersphere with a reasonable choice of N.

3. THE ISING MODEL

The Ising model is a model of magnetization of a material in which individual moments are represented by dipoles or "spins" arranged on a grid or lattice:

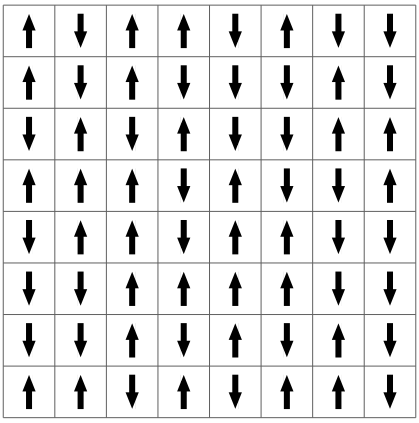


Figure 3: The model we are using in which the spins $s_i = \pm 1$, are restricted to point in only two directions, up and down, on the points of the lattice, where $s_i = +1$ for up-pointing spins and $s_i = -1$ for down-pointing ones.

in which the total magnetization of a material is reflected by how many spins are aligned with one another.
In the ferromagnetic case, the magnetic potential energy due to the interaction of two dipoles with spin s_i and s_j is given by: $e_{ij} = -Js_is_j$, where J is a positive-valued interaction constant. Normally it is assumed that spins interact only with those that are immediately adjacent to them on the lattice, which gives a total energy for the entire system equal to

$$E = -J \sum_{\langle ij \rangle} s_i s_j \,,$$

where the notation $\langle ij \rangle$ indicates a sum over adjacent pairs i, jon the lattice; on a square lattice each spin has four adjacent neighbors with which it interacts.
Write a program to perform a Markov chain Monte Carlo simulation of the Ising model on the square lattice for a system of 20×20 spins with $J = 1$ and temperature T in units where the Boltzmann constant k_B is also 1. Assume the initial magnetization is roughly zero – the initial spins are randomly distributed such that about half of them are up and half down. Your program should:

a. Include a function to calculate the total energy of the system, as given by the equation above.

b. Use your function as the basis for a Metropolis-style simulation of the Ising model that iterates by choosing a spin at random, flipping it, calculates the change in energy as a result of the flip, and deciding whether to accept the flip using the Metropolis acceptance formula, Newman's Eq. 10.60.

c. Run a set of simulations for $T = 1, 2, 3$, each for a million Monte Carlo steps. Make a plot of the total magnetization $\sum_i s_i$ of the system relative to the maximum possible as a function of steps taken for each run of the simulation.

d. For each simulation run with a different temperature, include a subplot of the lattice spin states as a density plot at the end of the simulation. Save the final 4 panel plot as `ising_[LASTNAME].png`.

Hint: It is best to try to figure out how to do this as an array operation rather than iterating through the grid

4. GLOBAL MINIMUM OF A FUNCTION

Write a program using simulated annealing to calculate the global minimum of a function $f(x)$. Start with $x = x_0$ with Monte Carlo moves of the form $x \to x + \delta$ where δ is a random number drawn from a Gaussian distribution with a mean of zero and standard deviation equal to one. You program should:

a. Include a function to calculate the global minimum given a function $f(x)$ and a starting point x_0. The function should use an exponential cooling schedule and allow you to adjust the start and end temperatures, as well as the exponential constant.

b. Consider the function $f(x) = x^2 - \cos 4\pi x$, whose global minimum occurs at $x = 0$. With a starting point $x_0 = 2$, validate and calibrate your function against the expected result, choosing the exponential cooling parameters until you retrieve a good result within a reasonable run time.

c. Make a plot of the values of x as a function of time during the run. Save this figure as `min1_[LASTNAME].png`.

d. Adapt your program to calculated the global minimum for the more complicated function $f(x) = \cos x + \cos \sqrt{2}x + \cos \sqrt{3}x$ over the range $0 < x < 50$. Note that there are several potential minima that may be found. Run your simulated annealing function several times and use the results to calculate and print the true global minimum.

5. DIFFUSION-LIMITED AGGREGATION

Consider a model for *diffusion-limited aggregation* (DLA): starting with a single particle in the middle of a square grid, the particle performs a random walk from square to square on the grid until it reaches a point on the edge of the system, at which point it "sticks" to the edge, becoming anchored there and immovable: Then a second particle starts at the

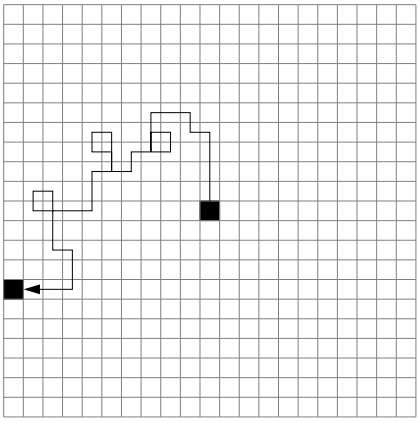


Figure 4: At each step, it is decided whether the particle moves either up, down, left, or right.

center and does a random walk until it sticks either to an edge or to the other particle, iterating each subsequent particle as starting at the center and walking until it sticks either to an edge or to any anchored particle.

Write a program to compute the positions of particle undergoing DLA on a 101×101 lattice, in which the position of each particle is represented by (i, j) with $(0, 0)$ as the central square of the lattice. Your program should:

a. Include a function that updates a data structure for storing positions of the anchored particles and an algorithm to check a particle's neighboring squares to see if they are outside the edge of the system or are occupied by an anchored particle.

b. The function should include an end condition to stop running once there is an anchored particle in the center of the grid (at this point any new particles would be immediately anchored anyway).

c. Visualize the results of the function to show how the distribution of anchored particles evolves over the course of the simulation. You may find it effective to map the color of the particles according to their relative age, changing gradually from the first particle added to the last.

If you are feeling patient, you can try to make the visualization for a 201×201 lattice for more impressive visuals

d. Save this visualization as `DLA_[LASTNAME].png`.